

TaskOrbit User Documentation

What is TaskOrbit

TaskOrbit is a conversational AI platform designed to handle multi-step tasks reliably over voice and chat. Traditional voice agents fail in production because their behaviour is governed by a single, monolithic system prompt that grows unmanageable as requirements expand — causing instruction drift, inconsistent responses, and slow iteration cycles. TaskOrbit replaces the single giant prompt with modular, JSON-based Task Definitions. A lightweight orchestration engine drives the conversation step by step, loading only the context relevant to the current moment. The result is an agent that behaves predictably, is easy to update, and can be composed from reusable building blocks without rewriting infrastructure.

How TaskOrbit Works

Key Concepts

Concept	Description
Task Definition	A JSON file describing one complete agent workflow (e.g. “book an appointment”)
Workflow Step	A single action within a task (e.g. “greet”, “collect data”, “confirm”, “end call”)
Agent	A configured identity — persona, instructions, pipeline config, and tools — defined in the task JSON
Session	One conversation run, holding the current step index and all collected data
Orchestration Engine	The Python component that reads the task JSON and drives the conversation step by step
Tool	A callable action the agent can trigger (e.g. extract data, end call, transfer to another agent)

Getting Started

Prerequisites

Docker Desktop is the only required local installation. Python, Poetry, Node.js, and PostgreSQL all run inside containers and do not need to be installed on the host.

Running the Full Stack

1. Clone the repository.
2. Copy the environment template files:

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```
3. Start the stack:

```
docker compose up --build
```

4. This starts four services:

- PostgreSQL on port 5432
- FastAPI backend on port 8000
- React frontend on port 5173
- LiveKit voice worker (internal port, proxied through the backend)

5. Open <http://localhost:5173> in your browser.

The first run takes several minutes while Docker pulls and builds base images. Subsequent starts are fast because layers are cached.

Using the Agent

Example: A Complete Agent

The canonical example bundled with TaskOrbit is a front-desk receptionist for Mueller Plumbing that books service appointments over voice. The workflow has four steps:

1. **Greet** — the agent delivers a first message directly, with no LLM call, so the response is instant.
2. **Collect** — the agent gathers three required inputs: caller name, phone number, and preferred service date.
3. **Confirm** — the agent reads back the collected details and asks the caller to confirm.
4. **End call** — the agent closes the session.

For programmatic access to this workflow and others, see the API documentation at <http://localhost:8000/docs>.

Agent Configuration

Agents are defined by JSON task definitions managed through the configuration UI at <http://localhost:5173/config> or via the API at `/v1/agent-configs`.

Voice Interface

1. Open <http://localhost:5173>.
2. Click **Start Session** — the browser will request microphone access.
3. The agent delivers its greeting; the microphone is automatically enabled after the greeting plays.
4. Speak naturally. A turn commits automatically after approximately two seconds of silence. You can also click **Send** to commit the turn immediately.
5. The agent's response is synthesized via ElevenLabs and plays through your browser speakers.
6. To interrupt the agent mid-response, start speaking — barge-in detection cuts the agent's audio immediately and begins processing your new turn.
7. Click **End Call** to terminate the session.

Persona Guardrails

Each agent can define an explicit operating scope via `persona_constraints` in its configuration. This block specifies the agent's permitted topics and a list of out-of-scope categories — for example,

medical advice or legal advice — along with a configured refusal template. The orchestration engine enforces this constraint at the system-prompt level: any user request that falls outside the defined scope triggers the refusal message automatically, without requiring the LLM to reason about whether to comply. This makes the boundary consistent and auditable.

API Access

TaskOrbit exposes a REST API for programmatic use. The base URL when running locally is <http://localhost:8000>. Interactive documentation with request/response schemas and a built-in request runner is available at <http://localhost:8000/docs>.

Key endpoints:

Method	Path	Purpose
POST	<code>/v1/conversations/process</code>	Submit a conversation turn and receive the agent's reply
GET	<code>/v1/agent-configs</code>	List all saved agent configurations
GET	<code>/v1/agent-configs/{id}</code>	Load a specific saved configuration
POST	<code>/v1/agent-configs</code>	Save a new agent configuration
POST	<code>/v1/livekit/token</code>	Mint a LiveKit room token for a voice session
GET	<code>/health</code>	Service health check

All endpoints are documented with full request and response schemas, field descriptions, and curl examples at <http://localhost:8000/docs>.